

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I VIJAY KARTHIKKEYAN(192511348)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments

- iii. Describe how the rover builds and updates its internal model
- iv. Compare online search with uninformed and informed search strategies
- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

1. AI-Powered Drone for Emergency Medicine Delivery

i. Greedy Best-First Search (GBFS):

Greedy Best-First Search chooses the path that appears closest to the destination using only the heuristic value $h(n)$ (straight-line distance). It ignores the actual travel cost.

Behavior in this scenario:

- The drone always selects the path that seems nearest to the destination.
- It makes quick decisions using the heuristic.
- If a road becomes blocked or weather changes, the drone may need to restart or change its route.

Strengths

- Fast decision-making.
- Uses less memory.
- Suitable when an immediate response is needed.

Weaknesses

- Does not guarantee the shortest or safest route.
- Can choose risky paths because it ignores travel cost.
- Performance decreases when the heuristic is inaccurate.

ii. A* Search

A* uses both:

- $g(n)$: Actual cost from the start to the current node.
- $h(n)$: Estimated cost from the current node to the goal.

Formula:

$$f(n) = g(n) + h(n)$$

Behavior

- Considers both travel cost and estimated remaining distance.
- Avoids expensive or dangerous routes.
- Updates its search when movement costs change.

Advantages

- Finds the optimal route when the heuristic is admissible.
- Produces safer and more reliable paths.
- Balances speed and cost effectively.

iii. Impact of Heuristic Accuracy

For Greedy Best-First Search

- Accurate heuristic → Very fast and efficient.
- Poor heuristic → May select dangerous or very expensive paths.

For A*

- Accurate heuristic → Faster search with optimal results.
- Poor heuristic → More nodes are explored, but optimality is still maintained if the heuristic never overestimates.

iv. Effect of Dynamic Changes

Blocked roads and changing weather require the algorithm to re-plan.

Greedy Best-First Search

- May repeatedly choose blocked paths.
- Requires frequent restarting.
- Less reliable.
- Updates path costs.
- Finds an alternative route.

- Handles changing environments more effectively.

v. Recommendation

Recommended Algorithm: A*

Justification

- Produces optimal routes.
- Considers both distance and travel cost.
- Safer in emergency situations.
- Adapts better to blocked paths and weather changes.
- Although slightly slower than GBFS, its reliability makes it the best choice.

2. Smart City Traffic Signal Optimization

Problem Formulation

Objective

Minimize:

- Total waiting time
- Fuel consumption
- Traffic congestion

Variables

Traffic signal timings at each intersection.

Constraints

- Hundreds of intersections.
- Continuously changing traffic.
- Very large search space.

Why Traditional Search is Inefficient

Traditional search explores every possible signal combination.

Since the number of intersections is huge, the search space grows exponentially, making exhaustive search computationally impossible.

i. Hill Climbing

Hill Climbing starts with one solution and repeatedly makes small improvements.

Example:

- Increase green-light duration.
- Measure traffic improvement.
- Keep the better solution.

Limitations

- Can stop at local optimum.
- Cannot guarantee the global optimum.
- Sensitive to the starting solution.

ii. Local Maxima, Plateaus and Ridges

Local Maximum

A signal timing that looks best locally but is not the overall best.

Plateau

Many neighboring solutions have the same performance.

The algorithm cannot determine which direction to move.

Ridge

Improvement requires moving in multiple directions simultaneously.

Hill Climbing struggles because it changes only one step at a time.

iii. Simulated Annealing and Genetic Algorithms

Simulated Annealing

- Sometimes accepts worse solutions.
- Escapes local maxima.
- Eventually settles on a better solution.

Genetic Algorithm

Works like natural evolution.

Steps:

1. Generate multiple traffic plans.
2. Select the best plans.
3. Combine them.
4. Apply random mutations.
5. Repeat until an excellent solution is found.

Advantages:

- Explores many solutions simultaneously.
- Works well for large optimization problems.
- Highly scalable.

iv. Recommendation

Recommended Algorithm: Genetic Algorithm

Reason

- Handles very large search spaces.
- Adapts continuously.
- Escapes local optima.
- Suitable for dynamic traffic systems

3. Autonomous Mars Rover

i. Why Online Search?

The rover does not know the complete environment before starting.

As it moves:

- New obstacles appear.
 - New terrain is discovered.
 - Decisions must be made immediately.
- Therefore, online search is required.

ii. Challenges

Partial Observability

- Only nearby terrain is visible.
- Hidden hazards may exist.

Dynamic Environment

- Dust storms.
 - Loose rocks.
 - Sensor uncertainty.
- These require continuous adaptation.

iii. Internal Model

The rover continuously updates its map.

Steps:

1. Sense surroundings.
2. Detect obstacles.

3. Store discovered locations.
 4. Update safe paths.
 5. Plan the next move.
- Its internal map becomes more accurate over time.

iv. Comparison

Online Search	Uninformed Search	Informed Search
Learns while moving	Requires complete map	Uses heuristic
Suitable for unknown environments	Poor for unknown terrain	Better than uninformed but still assumes known information
Highly adaptive	Less adaptive	Moderately adaptive

v. Recommendation

Recommended Approach: Online Search Agent

Reasons:

- Handles uncertainty.
- Continuously updates knowledge.
- Safer navigation.
- Suitable for unknown environments.

4. University Exam Timetable (CSP)

CSP Formulation

Variables

Each course exam.

Example:

- Math
- Physics
- AI
- Database

Domains

Possible:

- Date
- Time slot
- Exam hall

Constraints

1. No overlapping exams for students.
2. Limited exam halls.
3. Subject ordering requirements.
4. Faculty availability.
5. Special accommodations.

i. Variables, Domains and Constraints

Variables:

Each exam.

Domains:

Available dates, halls and time slots.

Constraints:

Must satisfy all scheduling rules without conflicts.

ii. Backtracking Search

Steps:

1. Assign the first exam.
2. Assign the next exam.
3. If a conflict occurs, undo the assignment.
4. Try another option.
5. Continue until all exams are scheduled.

iii. Constraint Propagation (Forward Checking)

Forward checking removes invalid choices before they are selected.

Example:

If Hall A is assigned at 10 AM, that option is removed from other exams.

Benefits:

- Fewer conflicts.
- Faster search.
- Less backtracking.

iv. Recommendation

Best Strategy

- Backtracking + Forward Checking + Constraint Propagation.

Reasons:

- Efficient.
- Reduces unnecessary search.
- Scales well to many courses and students.

5. Strategic Game AI (Minimax & Alpha-Beta Pruning)

i. Minimax Algorithm

Minimax assumes:

- AI tries to maximize its score.
- Human player tries to minimize the AI's score.

The AI evaluates every possible move and chooses the one with the best worst-case outcome.

ii. Computational Challenges

Game trees become extremely large.

Problems:

- Millions of possible moves.
- High memory usage.
- Slow decision-making.
- Difficult to meet real-time deadlines.

iii. Alpha-Beta Pruning

Alpha-Beta Pruning skips branches that cannot improve the final decision.

Advantages:

- Explores fewer nodes.

- Produces the same optimal result as Minimax.
- Significantly faster.
- Allows deeper search within limited time.

iv. Evaluation Function

The evaluation function assigns a score to each game state.

Factors:

- Resource availability.
- Army strength.
- Territory controlled.
- Defensive position.
- Distance to objectives.
- Remaining health.
- Strategic advantage.

A higher score indicates a better position for the AI.

v. Recommendation

Recommended Strategy: Alpha-Beta Pruning with Minimax

Justification

- Maintains optimal decision quality.
- Greatly reduces computation.
- Meets real-time decision requirements.
- Widely used in game AI because it balances accuracy and speed.